

摘要：

OpenAI联合创始人Andrej Karpathy指出，去年12月是AI范式转变的关键节点，以代理为核心的"软件3.0"时代已实质到来。他区分"氛围编程"与"代理工程"两个概念，强调后者能实现远超10倍的生产力提升。面对高度自动化未来，人类核心价值将回归审美、判断力与深层理解，"你可以外包思考，但无法外包理解"。

OpenAI联合创始人Andrej Karpathy在最新访谈中指出，大语言模型正作为“新型计算机”全面重塑计算架构。

4月29日，曾一手主导特斯拉Autopilot开发、并在OpenAI具有举足轻重地位的AI领军人物Andrej Karpathy，在由AI Sent主办的活动上，就当前AI代理的技术跃迁及其对软硬件生态的深远影响进行了深度拆解。



Karpathy表示，自去年12月起，他开始意识到以代理为核心的工作流已真正可用，这一转变标志着Software 3.0时代的实质到来。他说：

很多人去年对AI的印象还停留在ChatGPT，但你必须重新审视，尤其是从12月开始——事情已经发生了根本性变化。

他同时提出了“代理工程”（agentic engineering）这一新概念，以区别于去年他所命名的“氛围编程”（vibe coding），前者指的是专业软件开发中质量标准的延续与加速。

他直言，大量现有代码和应用程序在新范式下“不应该存在”，而当前多数组织的招聘流程、开发工具和基础设施，仍在为人类而非代理而设计。

软件3.0的黎明：底层计算架构的权力交接

科技产业正站在一个从量变到质变的十字路口。

去年12月是一个关键的转折点，Karpathy坦言，面对最新的AI模型，他经历了深刻的震撼：

系统生成的代码块越来越完美，我甚至记不清上一次修改它是什么时候。我只是越来越信任这个系统……（这让我）从未感觉自己作为一名程序员如此落后。

这种冲击是计算范式的彻底颠覆。在Karpathy看来，市场目前低估了这一变化的深度。

他指出，我们正在告别“软件1.0（写代码）”和“软件2.0（整理数据集训练神经网络）”，正式迈入“软件3.0”时代。

在这个新纪元里，大语言模型本身就是一台“新型计算机”。他说：

你现在的编程变成了写提示词，而在上下文窗口中的内容，就是你操控那个作为解释器的大语言模型的杠杆，让它在数字信息空间中执行计算。

更令市场瞩目的，是他对未来底层硬件架构演进的大胆预测。目前，神经网络仍以虚拟化的形式运行在现有的计算机上，但他认为未来这种主客关系将会反转：

你可以想象，神经网络将成为主进程，而CPU则变成了某种协处理器。神经网络将承担绝大部分的繁重工作。

这意味着，主导全市场资本开支的“智能算力”，其战略核心地位将在未来进一步固化。

下一代基建：重构“代理原生”生态

当执行和编码被机器接管，人类的核心价值和未来的基建形态将走向何方？

Karpathy直言：

所有的东西都必须被重写。

目前互联网的各种框架和库的说明文档仍是“为人类编写的”，这让他感到无比烦恼。Karpathy抱怨道：

为什么还要告诉我怎么做？我什么都不想做。我应该复制粘贴什么文本给我的AI代理？

未来的市场大机会在于构建“代理优先”的基础设施。

在这个世界里，系统被拆解为感知世界的“传感器”和改造世界的“执行器”，数据结构要让大语言模型高度可读，机器代理代表个人和机构在云端进行交互。

在如此高度自动化的未来，人类的核心稀缺性将回归到审美、判断力以及最深层的商业理解上。

Karpathy引用了一句令他反复咀嚼的话作为总结：



你可以外包你的思考，但你无法外包你的理解。

代理工程：远超“10倍工程师”的产能大爆发

在提升生产力这一市场最关心的维度上，Karpathy区分了两个核心概念：“氛围编程（Vibe coding）”与“代理工程（Agentic engineering）”。

他指出，“氛围编程”抬高了全员开发软件的下限，而“代理工程”则旨在维持专业软件的质量上限。

“代理工程”不仅仅是提速，它要求开发者协调那些“有些易错、带有随机性但极其强大”的AI代理，在不牺牲质量的前提下全速前进。

这也将极大拓宽企业产出的想象空间。Karpathy指出：

人们以前常谈论‘10倍工程师’，10倍已经不足以形容你所获得的提速了。在我看来，那些在这个领域表现出色的人，其产出峰值远超10倍。

面对这种生产力爆发，企业的组织架构和人才筛选逻辑必须重构。

他建议企业抛弃传统的算法解题面试，转而考察候选人如何利用多个AI代理协同构建大型项目，并能抵御其他AI代理的攻击。

AI商业落地的发力点

对于当前急于寻找AI应用落地场景的创业者和投资者，Karpathy提供了一个极具实操性的评估框架：可验证性。

目前AI的能力呈现出一种极其怪异的“锯齿状”。他举例表示：

最先进的模型今天能同时重构10万行的代码库或寻找零日漏洞，然而却告诉我应该步行去50米外的洗车店洗车，这简直疯了。

产生这种割裂的原因在于，前沿实验室（如OpenAI等）将海量强化学习资源倾注在了“数学”和“代码”等结果易于验证的领域。

因此，只要置身于结果可验证的商业场景中，AI就能发挥巨大威力。

Karpathy暗示，市场上仍存在大量高价值，但尚未被头部实验室重点关注的可验证强化学习环境，这正是创业公司进行微调（Fine-tuning）和商业变现的巨大蓝海。

以下为播客全文，AI辅助翻译：

主持人



我们非常荣幸迎来第一位特别嘉宾。他参与构建了现代人工智能，继而致力于解释现代人工智能，偶尔也会给它重新命名。他是OpenAI的联合创始人之一，正是在这间办公室里创立了这家公司；当年也正是他让特斯拉的自动驾驶系统真正跑了起来。他有一种罕见的天赋——能够让最复杂的技术变革听起来既通俗易懂，又顺理成章。大家都知道，他去年创造了“氛围编程（vibe coding）”这个词。但就在过去几个月里，他说了一句更令人震惊的话：他从未像现在这样觉得自己作为一名程序员已经落伍了。今天的对话就从这里开始。Andre，感谢你的到来。

Andrej Karpathy

你好，很高兴来这里为大家开个头。

主持人

就在几个月前，你说你从未像现在这样觉得自己作为程序员已经落伍了。从你口中听到这句话，着实令人吃惊。能跟我们聊聊这背后的感受吗？是兴奋，还是不安？

Andrej Karpathy

两者都有。首先，我想像很多人一样，在过去大约一年里，我一直在使用一些智能体工具，比如Claude Code之类的东西。它在处理代码片段方面表现很好，虽然有时会出错，需要手动修改，但总体上还是很有帮助的。

然后，去年十二月是一个明显的转折点。我当时在休假，有更多的时间，我想很多人也有类似的体验。我开始注意到，用最新的模型，输出的代码片段直接就是对的，我不断提更多要求，也依然是对的。我都记不清上次修正它是什么时候了。我越来越信任这个系统，然后就进入了“氛围编程”的状态。

我觉得那真的是一个非常鲜明的转变。我尝试在Twitter（也就是X）上强调这一点，因为很多人去年接触AI的方式还停留在使用ChatGPT这个层面，但你真的需要重新审视，尤其是以十二月为节点，因为事情已经发生了根本性的变化——尤其是在智能体工作流这个维度上，那时它才真正开始好用。从那以后，我就一头扎进了兔子洞，我的副业项目文件夹里塞满了各种稀奇古怪的东西，我就是一直在用AI写代码。大概是这么回事，十二月发生的。自那以后，我就一直在观察和思考这件事的影响。

主持人

你谈了很多关于“LLM是一台新型计算机”的理念——它不只是更好的软件，而是一种全新的计算范式。软件1.0是明确的规则，软件2.0是习得的权重，软件3.0就是现在这个阶段。如果这个框架是对的，那么一个团队在真正相信这件事的那一天，会有什么不同的做法？

Andrej Karpathy

对，确实如此。软件1.0阶段，我在写代码；软件2.0阶段，我实际上是通过构建数据集和训练神经网络来编程，编程的方式变成了整理数据集、设计目标函数和神经网络架构。

接下来发生的事情是：当你在足够大量的任务上训练这些GPT模型或大语言模型时，由于是在整个互联网上进行训练，模型必须同时完成数据集中所有任务，这使它在某种意义上成为了一台可编程的计算机。

软件3.0阶段，你的“编程”变成了“提示词工程”，而上下文窗口中的内容就是你操控解释器的杠杆——这里的解释器就是LLM，它解释你的上下文并在数字信息空间中执行计算。这大概就是这个转变的本质。

有几个例子让我对此有了更深切的体会，也许值得分享一下。

当OpenClaw发布时，你要安装它，通常会期望这是一个shell脚本——运行脚本来完成安装。但为了兼容众多不同的平台和不同类型的计算机，这类shell脚本往往会变得极其臃肿复杂。而OpenClaw的安装方式，实际上是复制一段文本交给你的智能体，由它来完成安装。这种方式强大得多，因为你工作在软件3.0的范式之下，不需要精确地逐一说明所有配置细节。智能体有自己的智能，它会理解指令，观察你的运行环境，采取智能行动让一切运转起来，并在循环中自主调试。这强大太多了。

还有一个更极端的例子，来自我构建MenuGen的经历。MenuGen的想法是：你去餐厅，他们递给你菜单，但通常没有图片，所以你根本不知道那些菜长什么样。我想做的是：拍一张菜单的照片，然后获取每道菜大概的样子。于是我用“氛围编程”构建了一个应用，可以上传照片，做一系列处理，部署在Vercel上，对菜单重新渲染，列出所有菜品，并调用图像生成模型对每个菜品名称进行OCR识别，再生成对应的图片呈现给用户。

后来我看到了这件事的软件3.0版本，彻底震惊了我：只需把照片交给Gemini，然后说“用Nana Banana把这些内容叠加到菜单上”。Nana Banana直接返回了一张图片——就是我拍的那张菜单的照片——但它在像素层面上直接把菜单上各道菜的图片渲染了出来。这让我大受震撼，因为我的整个MenuGen其实是多余的——它工作在旧范式里，那个应用根本不该存在。软件3.0范式要原始得多，神经网络承担了越来越多的工作，你输入的是图片，输出的也是图片，中间根本不需要任何应用层。

所以我认为，人们需要重新审视自己的思维框架，不要局限于现有的范式，不要只把它看作是现有事物的加速版。真正发生的是：有些全新的东西现在成为可能了。回到你说的编程问题，我认为这个问题本身也反映了一种旧有的思维方式——因为这不只是关于编程变得更快，而是更广泛意义上的信息处理现在都可以被自动化了，它不只关乎代码。

过去的代码是在结构化数据上运行的，你在结构化数据上编写代码。但比如我的“LLM知识库”项目，基本上是让LLM为你的组织或个人生成wiki——这不是一个程序，是过去不可能存在的东西，因为没有代码能够基于一堆事实凭空生成一个知识库。但现在你可以把这些文档输入进去，以不同的方式重新编译、重新排序，从而创造出新的、有价值的内容——这是对数据的一种重新诠释。这些都是以前不可能实现的新事物。所以我一直想回归这样一个问题：不只是什么事情现在可以做得更快了，还有什么以前根本不可能实现的新机会。我甚至觉得后者更令人兴奋。

主持人

我非常喜欢你描绘的MenuGen进化路径和那种对比，我相信很多人也关注了你从去年十月到今年一二月的编程历程。如果继续推演下去，对标九十年代构建网站、二十年代构建移动应用、上一个云时代构建SaaS的历史节点，什么是今天大部分还未被构建、但事后回望会觉得理所当然的东西？

Andrej Karpathy

顺着MenuGen的例子说下去，很多代码其实都不应该存在，神经网络承担了绝大部分的工作。我确实觉得这条外推曲线会变得非常奇异。

可以想象，在某种意义上，完全的神经计算机是可能的——想象一种设备，它接收原始视频和音频，输入到本质上是神经网络的系统中，通过扩散模型渲染出一个界面，那个界面在某种意义上是为那个独特的时刻量身定制的。

在计算机诞生的早期，人们对计算机究竟会长成什么样子其实是困惑的——是像计算器，还是像神经网络？在五六十年代，这并不显而易见。当然，我们走了计算器这条路，建立了经典计算体系，而神经网络目前是虚拟运行在现有计算机之上的。但可以设想，未来这一切会翻转——神经



网络成为宿主进程，CPU成为协处理器。我们已经看到那张图表了，神经网络的算力需求将会超越并成为浮点运算消耗的主导。

所以你可以想象一个非常陌生、非常异类的未来形态：神经网络承担绝大多数重活，工具调用只是作为某些确定性任务的历史遗留附件。真正在主导一切的，是以某种方式相互联结的神经网络群。这个外推的终点可能会极其陌生，但我想我们大概率是一步一步走到那里的。这条路怎么走，尚待观察。

主持人

我想聊聊"可验证性"这个概念——AI会在输出结果可以被验证的领域更快、更容易地实现自动化。如果这个框架是对的，什么工作将会以人们意想不到的速度发生变化？有哪些职业，人们觉得是安全的，但实际上高度可验证？

Andrej Karpathy

我花了一些时间思考可验证性这个问题。传统计算机能轻松自动化的，是你能用代码明确描述的东西；而这一轮大语言模型能轻松自动化的，是你能够验证的东西。原因在于：前沿实验室在训练这些大语言模型时，是在构建巨大的强化学习环境，模型会被给予基于验证的奖励信号。正是因为这种训练方式，这些模型最终形成了一种"锯齿状"的能力图谱——在数学、代码等可验证领域能力极强，但在可验证性较差的领域则相对平淡、表现粗糙。

我写关于可验证性的文章，是想理解为什么这些模型的能力如此不均匀。这部分原因在于实验室训练模型的方式，但我认为也与实验室的侧重点有关——他们碰巧放入了哪些数据。有些东西在经济上更有价值，因此产生了更多的训练环境，因为实验室希望模型在那些场景下好用。代码就是一个典型的例子。很可能存在大量可验证的环境本可以被纳入训练，但因为它们的实用价值不够高，就没有进入数据集。

对我来说，一个最能说明"锯齿状智能"的经典例子曾经是："strawberry这个单词里有几个字母r？"模型以答错著称。现在的模型已经修正了这个问题，但新的例子来了：我想去一个50米外的洗车店洗车，我应该开车去还是走路去？当今最先进的模型会告诉你走路去，因为太近了。但问题是，你要去的是洗车店啊。

这是多么奇怪的事——最先进的Claude Opus 4.7可以同时重构十万行代码，或者发现零日漏洞，却告诉我走路去洗车店。这实在令人难以置信。

这种锯齿状的能力表明：第一，模型某些地方可能存在根本性的问题；第二，你还是需要参与其中，把它当成工具来使用，保持对它行为的一定掌控。所以我关于可验证性的所有写作，归根结底，是在试图理解为什么这些模型的能力是锯齿状的，是否有某种规律可循。我认为，答案是某种"可验证性"与"实验室关注度"的组合。

再分享一个很能说明问题的轶事：从GPT-3.5到GPT-4，人们注意到模型下棋的能力大幅提升。很多人以为这只是能力的自然演进，但实际情况是——这是公开信息，我在网上看到过——大量棋局数据进入了预训练集，仅仅因为数据分布的变化，模型的下棋能力就远超正常进步幅度。是OpenAI的某个人决定加入这些数据，于是这项能力就突然激增了。

这就是我为什么强调这个维度：我们在某种程度上是受实验室决策左右的，他们碰巧放进了什么，你就得到什么。你拿到的是一个没有说明书的东西，它在某些情况下好用，在某些情况下不好用，你需要去探索它。

如果你的应用恰好落在强化学习训练覆盖的回路上，你会如鱼得水；如果落在数据分布之外，你就会举步维艰。你需要搞清楚你的应用落在哪种情况，如果不在覆盖的回路里，就真的需要认真考虑微调，做一些自己的工作，因为指望大语言模型开箱即用是不现实的。



主持人

我想稍后再回到"锯齿状智能"这个话题。如果你今天是一位创始人，正在考虑创业，你找到了一个你认为可解决的问题，这个领域是可验证的。但你环顾四周发现，实验室已经在最显而易见的方向上——数学、代码等——获得了逃逸速度。你对在座的创始人有什么建议？

Andrej Karpathy

我想这可以回到上一个问题。可验证性让某件事在当前范式下变得可行，因为你可以向它注入大量强化学习。也许可以这样理解：即使实验室没有直接关注某个领域，这一点依然成立。如果你处于一个可验证的设置中，能够创建强化学习环境和数据样本，这实际上为你开辟了自己做微调的道路，你可能会从中获益。

这是一种在技术上确实可行的路径：如果你有大量多样化的强化学习环境数据，可以使用你喜欢的微调框架，拉下这个杠杆，得到表现相当不错的东西。我不想具体说是哪些例子，但我确实认为存在一些非常有价值的强化学习环境，还没有被纳入训练的.....

说到这里，我不想在台上故意卖关子，但确实存在一些这样的例子。

主持人

那反过来，什么事情仍然只是看起来可以被自动化，实际上距离实现还很远？

Andrej Karpathy

我确实认为，几乎所有事情最终都可以被设计成可验证的，只是有些容易有些难。即便是写作这类事情，也可以设想用一组LLM评委来打分，大概也能得到相当不错的结果。所以更多是难度的问题，而不是能否做到的问题。我认为，归根结底，一切都是可以被自动化的。

主持人

好的，那么去年你创造了"氛围编程"这个词。今天我们身处一个感觉更严肃、更严谨的工程化世界。你认为两者的区别是什么？你会把我们现在所处的这个阶段叫做什么？

Andrej Karpathy

我认为，氛围编程是关于提升所有人在软件方面的能力下限——下限整体上升，每个人都可以用氛围编程做任何事，这非常了不起。

而"智能体工程（agentic engineering）"是关于在此基础上维持专业软件原有的质量标准。你不能因为氛围编程而引入安全漏洞，你仍然和以前一样对自己的软件负责。但你能不能更快？剧透：可以。但怎么才能做到位呢？

当我把它称为"智能体工程"时，是因为我认为这确实是一门工程学科。你有这些智能体——它们是某种"锯齿状"的存在，有些不可靠，有些随机性，但极其强大。问题是如何在不牺牲质量标准的前提下，协调它们来提速。做好这件事，就是智能体工程的领域。

我把这两个概念看作是不同的：一个是关于提升下限，另一个是关于突破上限。我正在看到的是，智能体工程师的能力上限非常高。以前人们谈论"10倍工程师"，我认为现在被放大的远不止这个数。10倍根本不是你能获得的加速幅度，从我现在的视角来看，真正精通此道的人的产出，要远远超过10倍。

主持人



我很喜欢这个框架。去年Sam Altman来AI Sent时，说了一句让人印象深刻的话：不同世代的人使用ChatGPT的方式不同。三十多岁的人把它当谷歌搜索的替代品，而十几岁的年轻人把ChatGPT当作进入互联网的入口。在今天的编程场景下，这个类比是什么？如果我们观察两个人用OpenAI的Codex或Anthropic的Claude Code写代码，一个是普通水平，一个是真正的AI原生程序员，你怎么描述两者的差异？

Andrej Karpathy

我认为核心在于：尽可能充分地利用手头可用的工具，善用它们的所有功能，并且在自己的工作流程上持续投入。就像以前的工程师会把VIM或VS Code用到极致，现在就是把Claude Code或Codex用到极致。所以本质上就是在自己的工具设置上投入，并善用所有可用的工具。

说到这里，有个相关的想法值得一提。如果很多团队现在要招聘智能体工程师，我认为大多数人的招聘流程还没有为此进行相应的改变。如果你还在出谜题让人来解，那还是老范式。新的招聘流程应该是：给我一个大项目，看你能不能把它做好——比如写一个Twitter克隆，做得好、做得安全，然后让智能体在上面模拟用户活动，再用多个Codex实例来尝试攻击你部署的网站，如果攻破了就算失败。我想这大概就是未来的样子——观察候选人在这种情境下构建大型项目、综合运用工具的表现。

主持人

随着智能体能做的事情越来越多，你认为哪种人类技能会变得更 valuable 而不是更不值钱？

Andrej Karpathy

目前来说，智能体基本上还是"实习生"级别的存在——能力出众，但还不稳定。所以你仍然需要负责把握审美、判断力、品味，以及适度的监督。

我最喜欢的一个例子，非常能说明智能体的奇怪之处：在MenuGen里，用户用Google账号注册，但用Stripe账号购买积分——两者各有自己的邮箱。结果我的智能体在处理积分充值时，试图用Stripe的邮箱来匹配Google邮箱，因为没有持久化的用户ID，它就尝试用邮箱来关联两个账户。但问题是，用户完全可以在Stripe和Google用不同的邮箱，这样资金就无法关联到账户了。这种错误非常奇怪——为什么要用邮箱来做跨系统的身份关联？邮箱本来就可以是任意的，可以不一样。

这类错误正是智能体现在还会犯的：你需要亲自负责规格设计和整体规划。说到"计划模式"，它当然很有用，但我觉得这里有更普遍的道理：你需要和智能体一起设计一份非常详尽的规格说明，也许就是以文档的形式，然后让智能体来写，而你负责监督和把控顶层的架构决策，智能体负责底层的实现细节。

再比如，关于神经网络中的张量操作，PyTorch、NumPy、Pandas之间有大量细节——keepdims还是keepdim，dim还是axis，reshape还是permute还是transpose——我已经记不住这些了，因为不需要记。这些细节可以交给"实习生"，因为它们的记忆力极好。但你仍然需要理解，比如说，底层有一个张量，有一个视图，你可以操作同一份内存的不同视图，或者你可以拥有不同的存储——后者效率更低。你仍然需要理解这些事情的本质，这样才不会做出多余的内存拷贝之类的低效操作。

所以你负责品味、工程设计、架构，确保整体方向是对的，确保需求是正确的，确保比如"我们要用唯一的用户ID来关联所有数据"——这类设计决策是你来做的。工程师负责填空，这大概就是我们目前的处境。

主持人



你觉得这种品味和判断力随着时间推移会变得不那么重要吗？还是说，它的上限会持续提升？

Andrej Karpathy

我确实希望这方面能有所改善。目前它无法改善，我认为还是因为它没有被纳入强化学习——可能没有相应的美感奖励，或者现有的奖励不够好。

说实话，当我真正看代码的时候，有时会有点心惊肉跳——不是每次输出的代码都特别好，往往很臃肿，大量复制粘贴，有一些脆弱的抽象，虽然能跑，但真的很丑。

一个很能说明问题的例子是nanoGPT项目——我一直在尝试把LLM训练代码简化到极致。模型在这件事上非常糟糕，没有能力做到。我不断尝试提示大语言模型去进一步简化，但就是不行。你感觉自己完全在强化学习回路之外，明显是在硬推，根本不是那种行云流水的状态。

所以我认为，人类现在仍然是这方面的主导者，但从根本上说，并没有什么原则性的障碍阻止这一点改变，只是实验室还没有做到这件事而已。

主持人

我想回到“锯齿状智能”这个话题。你写过一篇很有启发性的文章，讲到“动物与幽灵”的对比——我们构建的不是动物，而是在召唤幽灵。这些幽灵是由数据和奖励函数塑造的锯齿状智能体，而非由内在驱动、乐趣、好奇心或赋权感所驱动——那些是进化出来的东西。这个框架为何重要？它对我们构建、部署、评估乃至信任这些系统的方式，究竟改变了什么？

Andrej Karpathy

我写这篇文章是因为我想搞清楚这些东西究竟是什么。如果你对它们有一个准确的认知模型，你就能更好地使用它们。我不确定这个框架是否有很强的实用性，可能多少有些哲学意味，但我觉得它的核心在于：接受这样一个事实——这些东西不是动物智能。如果你对它们大喊大叫，它们不会表现得更好或更差，毫无影响。这一切都只是统计模拟回路，基底是预训练——统计学，然后上面叠加强化学习。

也许这只是一颗心态——我带着什么样的心态去面对它，什么可能有效，什么可能无效，如何调整它。我不能说我已经总结出“这里有五条明确的结论可以让你的系统更好”，更多是对它保持一种审慎的态度，随着时间推移慢慢摸索。

主持人

那就是起点。好，你现在深度参与的是那种不只是聊天的智能体——它们有真实的权限，有本地上下文，能代表你采取行动。当我们都开始生活这样的世界里，这个世界会是什么样子？

Andrej Karpathy

我想在座很多人都对原生智能体环境感到兴奋。所有东西都必须被重写——目前一切在根本上都还是为人类设计的，需要重新迁移。我现在使用的各种框架和库，它们的文档在根本上仍然是写给人看的。这是我最大的抱怨：为什么还有人告诉我该怎么操作？我不想自己动手。我想知道的是：我应该把什么复制粘贴给我的智能体？每次看到“请访问这个URL”之类的指引，就觉得很别扭。

我想，每个人都在思考这样一个问题：如何把需要完成的工作流分解成对世界的感知和对世界的执行？如何让一切对智能体友好？基本上就是：首先面向智能体描述它，围绕对LLM高度可读的数据结构构建大量自动化。



我希望看到大量面向智能体的基础设施。以MenuGen为例，那篇博文里很大一部分的麻烦不是写代码本身，而是部署——我要跟各种服务打交道，配置DNS，在各个设置菜单里来回跳，非常烦人。我希望的是：我给一个LLM一条提示，它就把MenuGen构建出来，而且自动部署好，我不需要碰任何东西，它就在网上跑起来了。这大概是一个好的测试标准，用来判断我们的基础设施是否正在变得越来越对智能体友好。

最终，我认为我们正在走向一个世界，每个人和每个组织都有自己的智能体代理。我的智能体和你的智能体互相沟通，搞定会议细节之类的事情。我觉得大致的方向就是这样，在座的大家都对此都感到兴奋，这没错。

主持人

我非常喜欢"感知器与执行器"这个比喻，这个思路真的很有意思。好，我想以教育这个话题来结束，因为你大概是世界上最擅长把复杂技术概念讲清楚的人之一，也非常深思熟虑地思考过如何围绕这些内容设计教育。当智能变得廉价，在AI的下一个时代，什么依然值得深入学习？

Andrej Karpathy

最近有一条推文深深触动了，我几乎每隔一天就会想起它。大意是：你可以外包你的思考，但你无法外包你的理解。

主持人

这话说得真好。

Andrej Karpathy

是的，因为我仍然是这个系统的一部分，信息仍然需要进入我的大脑。我越来越感觉自己成了瓶颈——仅仅是"知道"这件事本身就成了瓶颈：我们为什么要构建这个？有什么价值？怎么指挥我的智能体？

所以我仍然认为，最终必须有某种力量来指导思考和处理，而那个力量在根本上仍然受到"理解"的约束。这也是我为什么对LLM知识库感到兴奋——因为它是一种帮我消化信息的方式。每当我看到对同一份信息的不同视角和切入方式，我都觉得自己获得了洞见。本质上这是一种针对固定数据的合成数据生成。我真的很享受这个过程：读一篇文章，它进入我的wiki，然后我去问各种问题，探索不同角度。

这些工具，在某种意义上是增强理解的工具，而理解仍然是一种瓶颈——正是因为没有理解，你就无法做一个好的"导演"。大语言模型本身当然不擅长理解，这仍然是你独有的核心能力。所以我认为，增强理解的工具，是极其有趣和令人兴奋的方向。

主持人

期待几年后再次相聚，到时候看看我们是否已经被彻底自动化出局，连理解本身也被接管了。非常感谢你的到来，Andre，真的很感激。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。



限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

164 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论